1 文件权限与用户身份

查找索引：权限与身份 1 | 文件读写与缓冲 2 | 链接与目录 3 | 进程 4 | 信号 5 | 作业线程与判断题 6。

文件与目录的权限

以下按普通 UNIX 权限模型判断；遇到 ACL、粘滞位、只读文件系统等额外条件，要另行检查。

权限	普通文件	目录
r	读取文件数据	读取目录项，列出文件名
w	修改文件数据	修改目录项；创建、删除、改名通常还需要 x
x	允许执行程序文件	按名字搜索、通过此目录；cd 需要 x

权限检查的顺序

先匹配所有者，再匹配文件组（含进程附加组），否则使用其他用户位。**匹配某一类但权限不足，不再退回其他类。**

访问路径需要对沿途目录具有搜索权限。仅列名字主要需要目录 r；ls -l 获取各项属性还需要目录 x。

创建文件：父目录 w+x。删除文件：父目录 w+x，通常不要求目标文件本身可读或可写；粘滞位限制见第 6 页。

文件所有者即使没有读写权限，通常仍能 chmod 修改模式位；‘自己读不了’不等于‘不能改权限’。

用户身份与权限检查

名称	含义与典型用途
RUID / RGID	实际用户 / 组身份；getuid/getgid 取得。access 默认按实际身份检查。
EUID / EGID	有效用户 / 组身份；geteuid/getegid 取得。普通 open 等访问通常按有效身份检查。
保存的设置用户 / 组 ID	exec 完成特殊身份调整后，复制有效 ID；用于以后按规则恢复有效身份。
SUID / SGID 位	程序文件上的模式位，不是另一种进程 ID。执行时可把有效 ID 设为程序文件所有者 / 组。
st_uid / st_gid	stat 中的文件所有者 ID / 文件组所有者 ID；新文件 st_uid 通常取创建进程 EUID。

SUID 的典型例子

李执行张拥有的 SUID 二进制程序：RUID=李，EUID=张，保存 ID=张。seteuid 可临时切回实际身份，再按保存 ID 恢复；不会在权限不足时自动切换。

若秘密文件仅张可读：该进程 access(path,R_OK) 可失败，而 open(path,O_RDONLY) 可成功。access 仅检查，不打开；不要用它保证随后的 open 必然成功。

chmod u+s prog 设置 SUID，u-s 取消；权限串所有者 x 的位置显示 s（同时可执行）或 S（没有该 x 位）。调用者仍须获准执行。Linux 对脚本通常忽略 SUID，nosuid 等也可使其失效。

2 文件读写 偏移量与缓冲

描述符与打开方式

fd 是进程内的非负整数编号。0 / 1 / 2 通常为标准输入 / 标准输出 / 标准错误，可被重定向，不永久绑定终端。open 返回最小空闲编号；close 后编号可复用。

O_RDONLY 只读，O_WRONLY 只写，O_RDWR 读写；O_CREAT 不存在则创建，O_TRUNC 按适用条件清空原文件，O_EXCL 配合 O_CREAT 要求新建。创建模式通常还要受 umask 限制。

接口	成功返回	要点
open(path,flags[,mode])	fd，允许为 0	失败 -1；O_CREAT 时通常提供 mode
read(fd,buf,n)	实际读到的字节数	最多 n；正数请求在 EOF 返回 0；失败 -1
write(fd,buf,n)	实际写入的字节数	请求 n，不保证一定写满；失败 -1
lseek(fd,off,whence)	新的绝对偏移量	失败 -1；不读写，不直接改变文件长度
close(fd)	0	失败 -1；释放描述符，不等于删除文件

读写位置与空洞

同一次 open 对应的读写共用偏移量。实际成功读写 n 字节，位置前进 n；默认写覆盖原字节，超出末尾才扩长。分别 open 同一文件通常有独立位置；fork 继承或 dup 得到的描述符可共享位置。

SEEK_SET：新位置=off；SEEK_CUR：新位置=当前位置+off；SEEK_END：新位置=文件长度+off。off 可为负，但普通文件结果不能为负。lseek(fd,0,SEEK_CUR) 可查询位置。

定位超过 EOF 不改变长度；之后写入才扩长，中间形成空洞。空洞读出值为 0 的字节 '\0'，**仍计入 read 返回字节数**，不是字符 '0'，也不是 EOF。空洞可不分配数据块，但整个文件并非不占任何空间。

read 从文件内读到 EOF 时停住；若位置原本就在 EOF 外，read 返回 0，但不会自动把位置拉回末尾。

O_APPEND：每次 write 前定位当时的 EOF，定位与本次写入在普通本地文件系统上作为原子步骤。lseek 不能使后续追加写覆盖开头；read 仍按当前位置读取。先 lseek 再 write 的两个步骤存在竞争窗口。

复制循环与两层缓冲

read 只更新本次实际读到的内存，不自动补 '\0' 或清空剩余数组。复制时写 n 字节，不是数组容量；通常还有最后一次 EOF 检查。下面示例假设 write 每次完整成功。

```
while ((n = read(0, buf, sizeof buf)) > 0)
    write(1, buf, n);
```

机制	何时向下一层提交
C 库全缓冲	缓冲区满、显式 fflush/fclose、正常 exit 等
C 库行缓冲	输出换行、缓冲区满、显式刷新等
C 库不带缓冲	不积攒这层输出数据；不代表内核也无缓存

通常 stdout 连接终端时行缓冲，重定向到普通文件时全缓冲；Linux 常见 stderr 默认不带缓冲（标准要求不是全缓冲）。printf 等操作 FILE * 流，read/write 操作整数 fd。

printf → C 库缓冲 → write → 内核缓存 → 磁盘。fflush(stream) 提交库缓冲，fflush(NULL) 刷新所有输出流；fclose 刷新并关闭流；fsync(fd) 同步内核中的文件修改。fflush、close 或普通 write 成功都不等于已保证落盘。

3 文件链接与目录遍历

文件状态与 inode

目录项把名字关联到 inode；文件内容和属性不由文件名本身决定。inode 编号只保证在同一文件系统内有意义；不同文件系统上相同编号不表示同一文件。

stat(path,&st) 获取目标属性；lstat 遇符号链接时获取链接本身；fstat(fd,&st) 通过已打开的描述符取属性。成功 0，失败 -1。查询属性不要求目标数据可读，但路径查找需要目录搜索权限。

常见字段：st_mode 类型与权限，st_uid/st_gid 所有者，st_size 逻辑长度，st_nlink 硬链接数，st_ino inode 号。用 S_ISREG/S_ISDIR/S_ISLNK 等宏判类型，不把整个 st_mode 直接等于类型常量。

比较	硬链接	符号链接
本质	给同一 inode 增加一个名字	独立对象，保存目标路径字符串
文件系统	通常不能跨文件系统	可以跨文件系统
目录	通常禁止用户创建目录硬链接	可以指向目录
删除原名字	其他硬链接仍可访问原文件	可能变成悬挂链接；相对目标按链接所在目录解析
属性查询	各名字指向相同文件属性	stat 查目标，lstat 查链接本身

unlink 与文件生命期

unlink(path) 去掉一个名字，减少硬链接计数；不会使其他硬链接失效，也不会使已经打开的 fd 立即失效。

最后一个硬链接已删除，但仍有进程打开文件：仍可通过现存 fd 访问；最后一个相关打开引用释放后才可回收数据。不能再通过被删掉的旧路径打开，除非路径后来重新指向其他对象。

常用临时文件思路：创建并打开后立即 unlink，用 fd 继续访问，进程退出关闭最后引用后由内核回收。

路径与目录函数

以 / 开始为绝对路径；其他是相对路径，从进程当前工作目录出发。’.’ 是当前目录，’..’ 是父目录；根目录的父目录仍为根。

chdir(path) 修改调用进程的当前目录，成功 0、失败 -1；getcwd(buf,size) 取得当前工作目录的绝对路径，成功返回指针、失败 NULL。目录改变不会自动改变其他普通进程的当前目录。

opendir(path) 返回 DIR *，失败 NULL；readdir(dirp) 返回下一目录项指针，到末尾或出错返回 NULL，用 errno 区分；closedir 关闭目录流。d_name 是名字，不是完整路径。

遍历顺序不保证按字典序；递归目录通常要跳过 ’.’ 和 ’..’。只拿名字不等于已获取完整文件属性，属性通常还需要 stat/lstat。

目录树遍历程序 dopath 的答题点

递归处理一个子目录到结束，再处理兄弟节点，是深度优先。回调 func（示例 myfunc）在节点处执行具体任务，例如按文件类型计数、报告错误。

dopath 的返回值通常传播回调或递归返回值：0 表示继续，非 0 表示停止 / 向上传播；**非 0 不一定只能表示系统错误，也可能是回调主动要求停止。**

fullpath 拼接 ’/’ 和子名字后递归；ptr[-1]=0 在原来斜线处放字符串结束符，恢复父路径，防止后续使用残留子路径。修改前后要按代码中 ptr 的具体位置判断。

用 lstat 可识别链接本身；若随意改 stat，就会跟随链接，改变统计对象，目录链接还可能导致重复遍历或环。FTW_F / FTW_D / FTW_DNR / FTW_NS 的意义按题中自定义宏和回调判断。

4 进程控制与退出

fork 与继承

程序是代码，进程是一次运行及其状态。PID 标识进程，PPID 标识父进程。fork 成功后父子都从调用之后继续；父进程得到子 PID，子进程得到 0；失败仅父进程返回 -1。**子进程的真实 PID 不等于 fork 在子进程中的返回值 0。**

父子通常有独立地址空间，初值相同，普通全局 / 静态 / 局部变量后续各改各的。不能因为变量是全局的，就说父子共享该变量。没有同步时，不假定谁先运行。

打开描述符被继承，指向同一打开文件描述，因而可共享偏移量和状态标志；关闭父进程的描述符副本，不等于关闭子进程副本。当前工作目录、用户身份、进程组等也被继承。

stdio 缓冲属于进程内存，也会被复制；fork 前未刷出的输出可能被父子各自 exit 再刷一次。fork 前 fflush 或子进程适当使用 _exit 可避免该类重复。

exec 与简易 Shell

exec 成功替换当前进程的程序映像，不是又创建一个进程；PID 不变，成功不返回。失败返回 -1，才会执行后续报错 / 退出语句。execlp 中 p 表示按 PATH 搜索。

execlp(buf,buf,(char *)0) 只提供一个 argv[0]，不会把 'ls -l' 自动拆成命令和参数；fgets 常保留 '\n'，执行前要去掉它。

exit(127) 是示例 Shell 在执行失败路径选的退出状态；exec 成功后执行的是新程序，不能断言子进程最后一定返回 127。

```
pid = fork();
if (pid == 0) {
    execlp(buf, buf, (char *)0);
    _exit(127);
}
waitpid(pid, &status, 0);
```

wait 与僵尸进程

wait(&status) 等任一子进程；waitpid(pid,&status,0) 中 pid>0 等指定子进程，pid=-1 等任一子进程，pid=0 等同进程组的子进程，pid<-1 按进程组选择。成功返回子 PID；失败 -1；WNOHANG 且暂无可报告的子进程状态时可返回 0。

status 是编码后的终止状态，不直接等于 exit 的参数。先 WIFEXITED(status)，再用 WEXITSTATUS(status)；信号终止用 WIFSIGNALED/WTERMSIG。

僵尸：子进程已终止，但父进程尚未取走退出状态；保留 PID 和少量内核记录，不再运行。避免方法：wait/waitpid（可在 SIGCHLD 路径回收）；按题意双重 fork 并妥善回收中间子进程；显式 SIG_IGN 或 SA_NOCLDWAIT 另有 POSIX 规则。

SIGCHLD 默认动作显示为忽略，**不等于显式设置 SIGCHLD 为 SIG_IGN 的回收语义**；只写一个不回收的处理函数，也不能消灭僵尸。

终止方式	C 库缓冲 / 清理	文件描述符
从 main 返回 / exit	正常终止；调用 atexit，刷新并关闭标准 I/O 流	进程终止时由内核关闭
_exit / _Exit	不执行 atexit，不刷新 C 标准 I/O 缓冲	仍会关闭，不是‘不关文件’
abort	异常终止，不调用 atexit；是否刷新不能概括成必然	终止时由内核清理

从普通子函数 return 只是返回调用者，不等于终止进程；只有从 main 返回才具有上述正常终止意义。exit 时 atexit 通常按注册的逆序调用。

5 信号与信号程序阅读

信号的基本状态

信号是异步事件通知。处理方式：SIG_DFL 默认、SIG_IGN 忽略、安装函数捕获。阻塞是暂缓递送，不等于忽略；已产生但尚未递送是未决。

SIGKILL、SIGSTOP 不能被捕获、忽略或阻塞。SIGINT 常由 Ctrl+C 产生，SIGCHLD 通知子进程状态变化，SIGALRM 用于闹钟，SIGABRT 用于 abort。kill 的名字不代表一定杀死，发送何种信号取决于参数。

普通信号通常不排队：同种信号已有一个未决时，再来多次通常合并；实时信号另有排队语义。不能把‘不排队’推广到所有信号。

接口 / 字段	含义
sigaction(signo,&act,&old)	设置新动作并可保存旧动作；act=NULL 只查询；成功 0、失败 -1
act.sa_handler	处理函数指针、SIG_DFL 或 SIG_IGN
act.sa_mask	处理函数执行期间额外阻塞的信号集合
act.sa_flags	SA_RESTART / SA_NODEFER / SA_RESETHAND 等控制选项
sigprocmask(how,&set,&old)	单线程情形修改屏蔽字；SIG_BLOCK 加入、SIG_UNBLOCK 移出、SIG_SETMASK 替换
sigemptyset / sigfillset	初始化空集 / 全集；sigaddset / sigdelset 加入 / 删除成员

可靠 signal 包装函数的原卷结论

没有 SA_NODEFER 时，进入处理函数会自动阻塞引发它的同种信号，另外还阻塞 sa_mask 中的信号；返回后恢复原屏蔽字。

没有 SA_RESETHAND 时，捕获后不会自动恢复默认处理方式。题中 sigemptyset(&act.sa_mask) 不代表连当前同种信号也不阻塞。

题中对 SIGALRM 不加 SA_RESTART，其他信号在支持时加 SA_RESTART；因此可重启的低速调用在对应条件下重启。未重启的中断调用通常以 -1 / EINTR 报告；不是所有调用都必受 SA_RESTART 控制。

子进程结束会产生 SIGCHLD；安装处理函数不会自动回收退出状态，通常还需要 wait/waitpid。包装 signal 成功返回旧处理函数，失败返回 SIG_ERR。

继承与处理未决信号

fork：继承信号处理方式和屏蔽字，但子进程未决信号集初始为空。exec：已捕获信号通常恢复默认，忽略方式通常保留，屏蔽字保留。

不执行异步处理函数而处理未决信号：将可忽略信号动作设为 SIG_IGN 可丢弃其未决实例；也可先阻塞相关信号，再用 sigwait 同步接收消费。不要说解除阻塞必然能无处理函数清除。

可重入与 abort 阅读题

可重入：上一次调用未完成又被进入，仍不破坏状态。共享可变静态数据、内部非重入状态、malloc/free 或标准 I/O 等是常见风险；‘线程安全’不等于‘异步信号安全’。printf 通常不宜在信号处理函数中使用。

abort 强制异常终止：不能靠忽略或阻塞 SIGABRT 阻止；若处理函数返回，需恢复默认动作并再次产生信号。默认动作已终止的路径不会继续执行后面的语句。

教材实现中 fflush(NULL) 刷新输出流，不是 fclose；再次刷新用于处理函数可能产生的新输出。分析两次 kill 要沿实际分支走，不能说每次都一定执行两次。

现代 glibc 自 2.27 起 abort 不刷新流；POSIX 允许相关实现差异。若考题要求解释给定旧实现，按题面控制流答，不把旧实现的 fflush 行为推广成所有 abort 的保证。

6 作业控制 线程与判断题

进程组 会话与终端

进程组是一组进程，常对应一项作业 / 管道；可向整个组发信号，waitpid 也可按组选择子进程。会话是若干进程组的集合，用于组织作业控制与控制终端。

同一会话最多一个前台进程组；终端生成的 Ctrl+C 等信号发送给前台进程组。后台读控制终端通常触发 SIGTTIN；不能笼统说后台绝对不能向终端输出。

线程与竞争条件

竞争条件：结果依赖多个执行流访问共享状态的具体时序。临界区：访问共享状态、需要协调互斥的代码；可用互斥锁等保护。信号可用于同步通知，但普通信号会合并，不能不加条件地当可靠计数器。

同一进程的线程共享信号处理方式；各线程有独立屏蔽字，可用 pthread_sigmask。进程定向信号可交给一个未阻塞它的线程；定向线程信号交给目标线程。

POSIX pthread_kill 指定调用进程内的线程；Linux 特有 tgkill 可指定进程内某一线程，权限允许时也可作用于其他进程。答旧卷‘能否向别的进程的指定线程发信号’时应说明接口和系统范围。

POSIX fork 后，子进程只保留调用 fork 的那个线程；父进程原有 5 个线程，也不是子进程继承 5 个。exec 不保留原线程集。

零散简答与判断题纠错

题目关键词	可直接查用的结论
UID 为 0 的用户	不同登录名可配置相同 UID，包括 0；内核按数字身份识别。‘最多只能有一个 UID 0 登录名’不成立。不要实际创建此类账号。
修改密码	/etc/passwd 保存登录名、UID/GID、主目录、Shell 等；常见 shadow 系统的口令哈希与老化信息在 /etc/shadow，不是明文口令。passwd 程序受控更新权限受限的密码记录。
时间类型	日历时间通常用 time_t；进程 CPU 时间及 times 相关计数用 clock_t。‘UNIX 所有时间都用 time_t’错误。
墙上时钟时间	实际经过的时间，含等待与被其他任务占用的时间；用户 CPU 时间在用户态执行，系统 CPU 时间在内核代表进程执行。不要把墙钟时间等同 CPU 时间。
Shell / 系统调用	Shell 读取并解释命令，可交互运行或读脚本；系统调用是内核服务接口。ISO C 规定库函数接口，库函数可能封装系统调用，二者不是同一分类。
粘滞位	旧 UNIX 对可执行文件的历史含义是保留正文映像以加速装入。现代目录上的 sticky bit 限制删除 / 改名：通常须是文件所有者、目录所有者或具特权者；仍要检查目录 w+x。
流的定向	区分字节定向与宽字符定向，不是‘UTF-8 字符都只占一字节’。初始未定向，首次相应 I/O 或 fwide 可定向；通常不能直接混用两类 I/O。

程序题先画 fd 对应表或父子进程树；逐步记返回值、偏移量、内容和缓冲。信号题再查动作、屏蔽字和 flags。

材料与核对依据

范围来自 2014、2015、2016、2023、2025 年现有期中材料；2025 年回忆不完整。教材版本影响页码，程序题以实际代码为准。

接口与差异核对：Linux man-pages 的 open/read/write/lseek、access/execve/seteuid、stat/unlink/readdir/getcwd、fork/wait/exit/_exit、signal/sigaction/signal-safety、setbuf/fflush/abort、credentials/times/fwide；shadow-utils 的 useradd。

在线手册入口：https://man7.org/linux/man-pages/ 内核缓存：https://www.kernel.org/doc/html/latest/mm/page_cache.html